

Linear regression

Trong chương này, chúng ta cùng làm quen với một trong những thuật toán machine learning cơ bản nhất—linear regression (hồi quy tuyến tính). Qua chương này, bạn đọc sẽ có cái nhìn ban đầu về việc xây dựng một hệ thống machine learning. Linear regression là một thuật toán supervised, ở đó quan hệ giữa đầu vào và đầu ra được mô tả bởi một hàm tuyến tính. Thuật toán này còn được gọi là *linear fitting* hoặc *linear least square*.

7.1 Giới thiệu

Xét bài toán ước lượng giá của một căn nhà rộng x_1 m², có x_2 phòng ngủ và cách trung tâm thành phố x_3 km. Giả sử ta đã thu thập được số liệu từ 1000 căn nhà trong thành phố đó, liệu rằng khi có một căn nhà mới với các thông số về diện tích x_1 , số phòng ngủ x_2 và khoảng cách tới trung tâm x_3 , chúng ta có thể dự đoán được giá y của căn nhà đó không? Nếu có thì hàm dự đoán $y = f(\mathbf{x})$ sẽ có dạng như thế nào. Ở đây, vector đặc trưng $\mathbf{x} = [x_1, x_2, x_3]^T$ là một vector cột chứa thông tin đầu vào, đầu ra y là một số vô hướng.

Một cách trực quan, ta có thể thấy rằng: (i) diện tích nhà càng lớn thì giá nhà càng cao; (ii) số lượng phòng ngủ càng lớn thì giá nhà càng cao; (iii) càng xa trung tâm thì giá nhà càng giảm. Dựa trên quan sát này, ta có thể mô hình quan hệ giữa đầu ra và đầu vào bằng một hàm tuyến tính đơn giản:

$$y \approx \hat{y} = f(\mathbf{x}) = w_1x_1 + w_2x_2 + w_3x_3 = \mathbf{x}^T \mathbf{w} \quad (7.1)$$

trong đó $\mathbf{w} = [w_1, w_2, w_3]^T$ là vector hệ số (hoặc trọng số—*weight vector*) ta cần đi tìm. Đây cũng chính là tham số mô hình của bài toán. Mối quan hệ $y \approx f(\mathbf{x})$ như trong (7.1) là một mối quan hệ tuyến tính.

Bài toán trên đây là bài toán dự đoán giá trị của đầu ra dựa trên vector đặc trưng đầu vào. Ngoài ra, giá trị của đầu ra có thể nhận rất nhiều giá trị thực dương khác nhau. Vì vậy, đây là một bài toán regression. Mối quan hệ $\hat{y} = \mathbf{x}^T \mathbf{w}$ là một mối quan hệ tuyến tính. Tên gọi *linear regression* xuất phát từ đây.

Chú ý:

1. y là giá trị thực của đầu ra (*ground truth*), trong khi $\hat{y}$ là giá trị đầu ra dự đoán (*predicted output*) của mô hình linear regression. Nhìn chung, y và $\hat{y}$ là hai giá trị khác nhau do có sai số mô hình, tuy nhiên, chúng ta mong muốn rằng sự khác nhau này rất nhỏ.
2. *Linear* hay *tuyến tính* hiểu một cách đơn giản là *thẳng, phẳng*. Trong không gian hai chiều, một hàm số được gọi là *tuyến tính* nếu đồ thị của nó có dạng một *đường thẳng*. Trong không gian ba chiều, một hàm số được gọi là *tuyến tính* nếu đồ thị của nó có dạng một *mặt phẳng*. Trong không gian nhiều hơn ba chiều, khái niệm *mặt phẳng* không còn phù hợp nữa, thay vào đó, một khái niệm khác ra đời được gọi là *siêu mặt phẳng* (*hyperplane*). Các hàm số tuyến tính là các hàm đơn giản nhất, vì chúng thuận tiện trong việc hình dung và tính toán.

7.2 Xây dựng và tối ưu hàm mất mát

7.2.1 Sai số dự đoán

Sau khi đã xây dựng được mô hình dự đoán đầu ra như (7.1), ta cần tìm một phép đánh giá phù hợp với bài toán. Với bài toán regression nói chung, ta mong muốn rằng sự sai khác e giữa giá trị thực y và giá trị dự đoán $\hat{y}$ là nhỏ nhất. Nói cách khác, chúng ta muốn giá trị sau đây càng nhỏ càng tốt:

$$\frac{1}{2}e^2 = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(y - \mathbf{x}^T \mathbf{w})^2 \quad (7.2)$$

ở đây ta lấy bình phương vì $e = y - \hat{y}$ có thể là một số âm. Việc sai số là nhỏ nhất có thể được mô tả bằng cách lấy trị tuyệt đối $|e| = |y - \hat{y}|$, tuy nhiên, cách làm này ít được sử dụng vì hàm trị tuyệt đối không khả vi tại mọi điểm, không thuận tiện cho việc tối ưu sau này. Hệ số $\frac{1}{2}$ sẽ bị triệt tiêu sau này khi lấy đạo hàm của e theo tham số mô hình $\mathbf{w}$.

7.2.2 Hàm mất mát

Điều tương tự xảy ra với tất cả các cặp (*input, output*) $(\mathbf{x}_i, y_i), i = 1, 2, \dots, N$, với N là số lượng dữ liệu quan sát được. Điều chúng ta mong muốn—trung bình sai số là nhỏ nhất—tương đương với việc tìm $\mathbf{w}$ để hàm số sau đạt giá trị nhỏ nhất:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (y_i - \mathbf{x}_i^T \mathbf{w})^2 \quad (7.3)$$

Hàm số $\mathcal{L}(\mathbf{w})$ chính là hàm mất mát của linear regression với tham số mô hình $\theta = \mathbf{w}$. Chúng ta luôn mong muốn rằng sự mất mát là nhỏ nhất, điều này có thể đạt được bằng cách tối thiểu hàm mất mát theo $\mathbf{w}$:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \mathcal{L}(\mathbf{w}) \quad (7.4)$$

$\mathbf{w}^*$ là nghiệm cần tìm, đôi khi dấu $*$ được bỏ đi và nghiệm có thể được viết gọn lại thành $\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmin}} \mathcal{L}(\mathbf{w})$.

Trung bình sai số

Việc lấy trung bình (hệ số $\frac{1}{N}$) hay lấy tổng trong hàm mất mát, về mặt toán học, không ảnh hưởng tới nghiệm của bài toán. Trong machine learning, các hàm mất mát thường có chứa hệ số tính trung bình theo từng điểm dữ liệu trong tập huấn luyện. Khi tính giá trị của hàm mất mát trên tập kiểm thử, ta cũng tính trung bình lỗi của mỗi điểm. Việc lấy trung bình này quan trọng vì số lượng điểm dữ liệu trong mỗi tập dữ liệu có thể thay đổi. Việc tính toán mất mát trên từng điểm dữ liệu sẽ hữu ích hơn trong việc đánh giá chất lượng mô hình sau này. Ngoài ra, việc lấy trung bình cũng giúp tránh hiện tượng tràn số khi số lượng điểm dữ liệu quá nhiều.

Trước khi đi xây dựng nghiệm cho bài toán tối ưu hàm mất mát, ta thấy rằng hàm số này có thể được viết gọn lại dưới dạng ma trận, vector, và norm như dưới đây:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (y_i - \mathbf{x}_i^T \mathbf{w})^2 = \frac{1}{2N} \left\| \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} - \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix} \mathbf{w} \right\|_2^2 = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}^T \mathbf{w}\|_2^2 \quad (7.5)$$

với $\mathbf{y} = [y_1, y_2, \dots, y_N]^T$, $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$. Như vậy $\mathcal{L}(\mathbf{w})$ là một hàm số liên quan tới bình phương của ℓ_2 norm.

7.2.3 Nghiệm cho bài toán Linear Regression

Nhận thấy rằng hàm mất mát $\mathcal{L}(\mathbf{w})$ có đạo hàm tại mọi $\mathbf{w}$ (xem Bảng 2.1). Vậy việc tìm giá trị tối ưu của $\mathbf{w}$ có thể được thực hiện thông qua việc giải phương trình đạo hàm của $\mathcal{L}(\mathbf{w})$ theo $\mathbf{w}$ bằng không. Thật may mắn, đạo hàm của hàm mất mát của linear regression rất đơn giản:

$$\frac{\nabla \mathcal{L}(\mathbf{w})}{\nabla \mathbf{w}} = \frac{1}{N} \mathbf{X}(\mathbf{X}^T \mathbf{w} - \mathbf{y}) \quad (7.6)$$

Giải phương trình đạo hàm bằng không:

$$\frac{\nabla \mathcal{L}(\mathbf{w})}{\nabla \mathbf{w}} = \mathbf{0} \Leftrightarrow \mathbf{X} \mathbf{X}^T \mathbf{w} = \mathbf{X} \mathbf{y} \quad (7.7)$$

Nếu ma trận $\mathbf{X} \mathbf{X}^T$ khả nghịch, phương trình (7.7) có nghiệm duy nhất $\mathbf{w} = (\mathbf{X} \mathbf{X}^T)^{-1} \mathbf{X} \mathbf{y}$.

Nếu ma trận $\mathbf{X} \mathbf{X}^T$ không khả nghịch, phương trình (7.7) sẽ vô nghiệm hoặc có vô số nghiệm. Lúc này, một nghiệm đặc biệt của phương trình có thể được xác định dựa vào giả nghịch đảo (pseudo inverse). Người ta chứng minh được rằng¹ với mọi ma trận $\mathbf{X}$, luôn tồn tại duy nhất

¹ Least Squares, Pseudo-Inverse, PCA & SVD (<https://goo.gl/RoQ6mS>)

một giá trị $\mathbf{w}$ có ℓ_2 norm nhỏ nhất giúp tối thiểu $\|\mathbf{X}^T \mathbf{w} - \mathbf{y}\|_F^2$. Cụ thể, $\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger \mathbf{X}\mathbf{y}$, trong đó $(\mathbf{X}\mathbf{X}^T)^\dagger$ là giả nghịch đảo của $\mathbf{X}\mathbf{X}^T$. Giả nghịch đảo của một ma trận $\mathbf{A}$ luôn luôn tồn tại, thậm chí cả khi ma trận đó không vuông. Khi $\mathbf{A}$ là vuông và khả nghịch thì giả nghịch đảo chính là nghịch đảo. Nghiệm của bài toán tối ưu (7.4) là

$$\mathbf{w} = (\mathbf{X}\mathbf{X}^T)^\dagger \mathbf{X}\mathbf{y} \quad (7.8)$$

Hàm số tính giả nghịch đảo của một ma trận bất kỳ có sẵn trong thư viện numpy.

7.2.4 Bias trick

Chú ý rằng quan hệ $y = \mathbf{x}^T \mathbf{w}$ là một quan hệ tuyến tính. Linear regression thường đề nói tới một quan hệ hơi phức tạp hơn một chút khi có sự xuất hiện của một số hạng tự do b :

$$y = \mathbf{x}^T \mathbf{w} + b \quad (7.9)$$

Mỗi quan hệ này còn được gọi là *affine*. Nếu $b = 0$, đường thẳng $\mathbf{y} = \mathbf{x}^T \mathbf{w} + b$ luôn đi qua gốc toạ độ. Việc thêm hệ số b vào sẽ khiến cho mô hình linh hoạt hơn một chút bằng cách bỏ ràng buộc đường thẳng quan hệ giữa đầu ra và đầu vào luôn đi qua gốc toạ độ. Đại lượng b còn được gọi là *bias*. Đại lượng này cũng có thể *học được* như vector hệ số $\mathbf{w}$.

Để ý thấy rằng, nếu coi $x_0 = 1$, ta sẽ có:

$$y = \mathbf{x}^T \mathbf{w} + b = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b x_0 = \bar{\mathbf{x}}^T \bar{\mathbf{w}} \quad (7.10)$$

trong đó $\bar{\mathbf{x}} = [x_0, x_1, x_2, \dots, x_N]^T$ và $\bar{\mathbf{w}} = [b, w_1, w_2, \dots, w_N]$. Nếu đặt $\bar{\mathbf{X}} = [\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \dots, \bar{\mathbf{x}}_N]$ ta sẽ có nghiệm của bài toán tối thiểu hàm mất mát:

$$\bar{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{2N} \|\mathbf{y} - \bar{\mathbf{X}}^T \bar{\mathbf{w}}\|_2^2 = (\bar{\mathbf{X}}\bar{\mathbf{X}}^T)^\dagger \bar{\mathbf{X}}\mathbf{y} \quad (7.11)$$

Kỹ thuật thêm một đặc trưng bằng 1 vào vector đặc trưng và ghép bias b vào vector hệ số $\mathbf{w}$ như trên còn được gọi là *bias trick*. Chúng ta sẽ còn gặp lại kỹ thuật này nhiều lần trong cuốn sách này.

7.3 Ví dụ trên Python

7.3.1 Bài toán

Xét một ví dụ đơn giản có thể áp dụng linear regression. Chúng ta cũng sẽ so sánh nghiệm của bài toán khi giải theo phương trình (7.11) và nghiệm tìm được khi dùng thư viện scikit-learn của Python.

Xét ví dụ về dự đoán cân nặng dựa theo chiều cao. Xét bảng cân nặng và chiều cao của 15 người trong Bảng 7.1. Dữ liệu của hai người có chiều cao 155 cm và 160 cm được tách ra làm test set, phần còn lại tạo thành training set.

Bài toán đặt ra là: liệu có thể dự đoán cân nặng của một người dựa vào chiều cao của họ không? (*Trên thực tế, tất nhiên là không, vì cân nặng còn phụ thuộc vào nhiều yếu tố khác nữa, thể tích chẳng hạn*). Có thể thấy là cân nặng thường tỉ lệ thuận với chiều cao (càng cao càng nặng), nên có thể sử dụng linear regression cho việc dự đoán này.

Bảng 7.1: Bảng dữ liệu về chiều cao và cân nặng của 15 người

Chiều cao (cm)	Cân nặng (kg)	Chiều cao (cm)	Cân nặng (kg)
147	49	168	60
150	50	170	72
153	51	173	63
155	52	175	64
158	54	178	66
160	56	180	67
163	58	183	68
165	59		

7.3.2 Hiện thị dữ liệu trên đồ thị

Trước tiên, chúng ta khai báo training data:

```
from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt
# height (cm), input data, each row is a data point
X = np.array([[147, 150, 153, 158, 163, 165, 168, 170, 173, 175, 178, 180, 183]]).T
# weight (kg)
y = np.array([ 49, 50, 51, 54, 58, 59, 60, 62, 63, 64, 66, 67, 68])
```

Các điểm dữ liệu được minh họa bởi các điểm màu đỏ trong Hình 7.1. Ta thấy rằng dữ liệu được sắp xếp gần như theo một đường thẳng, vậy mô hình linear regression sau đây có khả năng cho kết quả tốt:

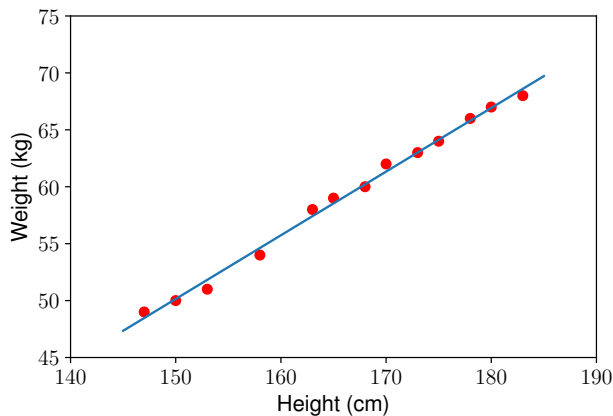
$$(\text{cân nặng}) = \mathbf{w}_1 * (\text{chiều cao}) + \mathbf{w}_0$$

ở đây $\mathbf{w}_0$ chính là bias b .

7.3.3 Nghiệm theo công thức

Tiếp theo, chúng ta sẽ tính toán các hệ số $\mathbf{w}_1$ và $\mathbf{w}_0$ dựa vào công thức (7.11). Chú ý: giả nghịch đảo của một ma trận $\mathbf{A}$ trong Python sẽ được tính bằng `numpy.linalg.pinv(A)`.

```
# Building Xbar
one = np.ones((X.shape[0], 1))
Xbar = np.concatenate((one, X), axis = 1) # each point is one row
# Calculating weights of the fitting line
A = np.dot(Xbar.T, Xbar)
b = np.dot(Xbar.T, y)
w = np.dot(np.linalg.pinv(A), b)
# weights
w_0, w_1 = w[0], w[1]
```



Hình 7.1: Phân bố của các điểm dữ liệu (màu đỏ) và đường thẳng xấp xỉ tìm được bởi linear regression.

Đường thẳng mô tả mối quan hệ giữa đầu vào và đầu ra được cho trên Hình 7.1. Ta thấy rằng các điểm dữ liệu màu đỏ nằm khá gần đường thẳng dự đoán màu xanh. Vậy mô hình linear regression hoạt động tốt với tập dữ liệu huấn luyện. Bây giờ, chúng ta sử dụng mô hình này để dự đoán dữ liệu trong test set:

```
y1 = w_1*155 + w_0
y2 = w_1*160 + w_0
print('Input 155cm, true output 52kg, predicted output %.2fkg' % (y1) )
print('Input 160cm, true output 56kg, predicted output %.2fkg' % (y2) )
```

Kết quả:

```
Input 155cm, true output 52kg, predicted output 52.94kg
Input 160cm, true output 56kg, predicted output 55.74kg
```

Chúng ta thấy rằng kết quả dự đoán khá gần với số liệu thực tế.

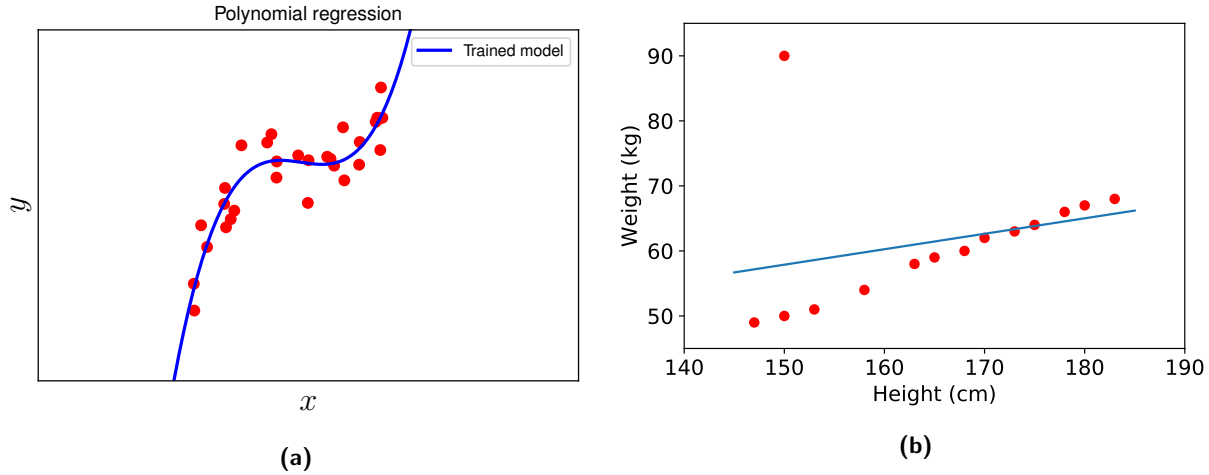
7.3.4 Nghiệm theo thư viện scikit-learn

Tiếp theo, chúng ta sẽ sử dụng thư viện scikit-learn để tìm nghiệm.

```
from sklearn import datasets, linear_model
# fit the model by Linear Regression
regr = linear_model.LinearRegression()
regr.fit(X, y) # in scikit-learn, each sample is one row
# Compare two results
print("scikit-learn's solution : w_1 = ", regr.coef_[0], "w_0 = ", regr.intercept_)
print("our solution : w_1 = ", w[1], "w_0 = ", w[0])
```

Kết quả dưới đây cho thấy rằng hai cách tính cho kết quả như nhau.

```
scikit-learn solution : w_1 = [ 0.55920496] w_0 = [-33.73541021]
our solution : w_1 = [ 0.55920496] w_0 = [-33.73541021]
```



Hình 7.2: (a) Polynomial regression bậc ba. (b) Linear regression nhạy cảm với nhiễu nhỏ.

7.4 Thảo luận

7.4.1 Các bài toán có thể giải bằng linear regression

Hàm số $y \approx f(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ là một hàm tuyến tính theo cả $\mathbf{w}$ và $\mathbf{x}$. Trên thực tế, linear regression có thể áp dụng cho các mô hình chỉ cần tuyến tính theo $\mathbf{w}$. Ví dụ,

$$y \approx w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 \sin(x_2) + w_5 x_1 x_2 + w_0 \quad (7.12)$$

là một hàm tuyến tính theo $\mathbf{w}$ và vì vậy cũng có thể được giải bằng linear regression. Với mỗi vector đặc trưng $\mathbf{x} = [x_1, x_2]^T$, chúng ta tính toán vector đặc trưng mới $\tilde{\mathbf{x}} = [x_1, x_2, x_1^2, \sin(x_2), x_1 x_2]^T$ rồi áp dụng linear regression với dữ liệu mới này. Tuy nhiên, việc tìm ra các hàm số $\sin(x_2)$ hay $x_1 x_2$ là tương đối *không tự nhiên*. *Hồi quy đa thức (polynomial regression)* thường được sử dụng nhiều hơn với các vector đặc trưng mới có dạng $[1, x_1, x_1^2, \dots]^T$. Một ví dụ về hồi quy đa thức bậc 3 được thể hiện trong Hình 7.2a.

7.4.2 Hạn chế của linear regression

Hạn chế đầu tiên của linear regression là nó rất **nhạy cảm với nhiễu** (*sensitive to noise*). Trong ví dụ về mối quan hệ giữa chiều cao và cân nặng bên trên, nếu có chỉ một cặp dữ liệu *nh nhiễu* (150 cm, 90kg) thì kết quả sẽ sai khác đi rất nhiều (xem Hình 7.2b).

Vì vậy, trước khi thực hiện linear regression, các nhiễu cần phải được loại bỏ. Bước này được gọi là *tiền xử lý (pre-processing)*. Hoặc hàm mất mát có thể thay đổi một chút để tránh việc tối ưu các nhiễu bằng cách sử dụng *Huber loss* (<https://goo.gl/TBUWzg>). Linear regression với Huber loss được gọi là *Huber regression*, được khẳng định là *robust to noise* (ít bị ảnh hưởng hơn bởi nhiễu). Xem thêm *Huber Regressor, scikit learn* (<https://goo.gl/h2rKu5>).

Hạn chế thứ hai của linear regression là nó **không biểu diễn được các mô hình phức tạp**. Mặc dù trong phần trên, chúng ta thấy rằng phương pháp này có thể được áp dụng nếu quan hệ giữa *outcome* và *input* không nhất thiết phải là tuyến tính, nhưng mỗi quan

hệ này vẫn đơn giản nhiều so với các mô hình thực tế. Hơn nữa, việc tìm ra các đặc trưng $x_1^2, \sin(x_2), x_1x_2$ như ở trên là ít khả thi.

7.4.3 Ridge regression

Trong trường hợp *ma trận $\mathbf{XX}^T$ không khả nghịch*, có một kỹ thuật nhỏ để tránh hiện tượng này là biến đổi $\mathbf{XX}^T$ một chút để biến nó trở thành $\mathbf{A} = \mathbf{XX}^T + \lambda \mathbf{I}$ với λ là một số dương rất nhỏ và $\mathbf{I}$ là ma trận đơn vị với bậc phù hợp.

Ma trận $\mathbf{A}$ là khả nghịch vì nó là một ma trận xác định dương. Thật vậy, với mọi $\mathbf{w} \neq \mathbf{0}$,

$$\mathbf{w}^T \mathbf{A} \mathbf{w} = \mathbf{w}^T (\mathbf{XX}^T + \lambda \mathbf{I}) \mathbf{w} = \mathbf{w}^T \mathbf{XX}^T \mathbf{w} + \lambda \mathbf{w}^T \mathbf{w} = \|\mathbf{X}^T \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 > 0 \quad (7.13)$$

Lúc này, nghiệm của bài toán là $\mathbf{y} = (\mathbf{XX}^T + \lambda \mathbf{I})^{-1} \mathbf{X} \mathbf{y}$. Nếu xét hàm mất mát

$$\mathcal{L}_2(\mathbf{w}) = \frac{1}{2N} (\|\mathbf{y} - \mathbf{X}^T \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2) \quad (7.14)$$

với phương trình đạo hàm theo $\mathbf{w}$ bằng không:

$$\frac{\nabla \mathcal{L}_2(\mathbf{w})}{\nabla \mathbf{w}} = \mathbf{0} \Leftrightarrow \frac{1}{N} (\mathbf{X}(\mathbf{X}^T \mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}) = \mathbf{0} \Leftrightarrow (\mathbf{XX}^T + \lambda \mathbf{I}) \mathbf{w} = \mathbf{X} \mathbf{y} \quad (7.15)$$

Ta thấy $\mathbf{w} = (\mathbf{XX}^T + \lambda \mathbf{I})^{-1} \mathbf{X} \mathbf{y}$ chính là nghiệm của bài toán tối thiểu $\mathcal{L}_2(\mathbf{w})$ trong (7.14). Mô hình machine learning với hàm mất mát (7.14) còn được gọi là *ridge regression*. Ngoài việc giúp cho phương trình đạo hàm theo hệ số bằng không có nghiệm duy nhất, ridge regression còn giúp cho mô hình tránh được overfitting như chúng ta sẽ thấy ở Chương 8.

7.4.4 Phương pháp tối ưu khác

Linear regression là một mô hình đơn giản, lời giải cho phương trình đạo hàm bằng không cũng khá đơn giản. *Trong hầu hết các trường hợp, chúng ta không thể giải được phương trình đạo hàm bằng không*. Tuy nhiên, nếu một hàm mất mát có đạo hàm không quá phức tạp, nó có thể được giải bằng một phương pháp rất hữu dụng có tên là gradient descent. Trên thực tế, một vector đặc trưng có thể có kích thước rất lớn, dẫn đến ma trận $\mathbf{XX}^T$ cũng có kích thước lớn và việc tính ma trận nghịch đảo có thể không lợi về mặt tính toán. Gradient descent sẽ giúp tránh được việc tính ma trận nghịch đảo. Chúng ta sẽ hiểu kỹ hơn về phương pháp này trong Chương 12.

7.4.5 Đọc thêm

1. *Simple Linear Regression Tutorial for Machine Learning* (<https://goo.gl/WRVda8>).
2. *Regularization: Ridge Regression and the LASSO* (<https://goo.gl/uRzN1K>).